K.R. MANGALAM UNIVERSITY

Sohna Road, Gurugram, Haryana

ASSIGNMENT - 2

COURSE

Deep Learning Architectures and Techniques (ETMMDL274)

SUBMITTED TO

Dr. Shahid Ahmad Wani
Faculty, Dept. of CSE/IT
K.R. Mangalam University

SUBMITTED BY

Raj Tilak Singh
Roll No: 2501940048
MCA (AI & ML) | Sem: 2nd

Department of Computer Science & Engineering

Academic Session: 2025 - 2027

Introduction

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for image classification and computer vision tasks. CNNs utilize convolution and pooling operations to automatically extract hierarchical spatial features from images.

In this experiment, a custom CNN architecture was developed for multiclass image classification using the CIFAR-10 dataset. Additionally, transfer learning was implemented using a pretrained MobileNetV2 model to compare performance. The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrix to analyze their effectiveness.

Dataset Description

The CIFAR-10 dataset was used for this experiment. It contains 60,000 color images of size 32×32 pixels across 10 classes. The dataset consists of:

- 50,000 training images
- 10,000 test images

Each image belongs to one of the 10 object categories.

The dataset is balanced, with each class containing an equal number of images. This makes it suitable for evaluating multiclass classification performance.

```
[1]
5m
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

tf.random.set_seed(42)
np.random.seed(42)

from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("Training shape:", x_train.shape)
print("Test shape:", x_test.shape)

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 321s 2us/step
Training shape: (50000, 32, 32, 3)
Test shape: (10000, 32, 32, 3)
```

Figure 1: CIFAR-10 Dataset Shape

Data Preprocessing

The following preprocessing steps were applied:

- Pixel values were normalized to the range [0,1].
- Labels were converted into one-hot encoded format.
- Random seed was fixed to ensure reproducibility.

Data normalization improves training stability and convergence speed.

Preprocessing ensures faster convergence and improves overall model performance during training.

Data augmentation techniques such as random horizontal flipping, rotation, and zoom were applied to reduce overfitting and improve generalization.

Custom CNN Architecture

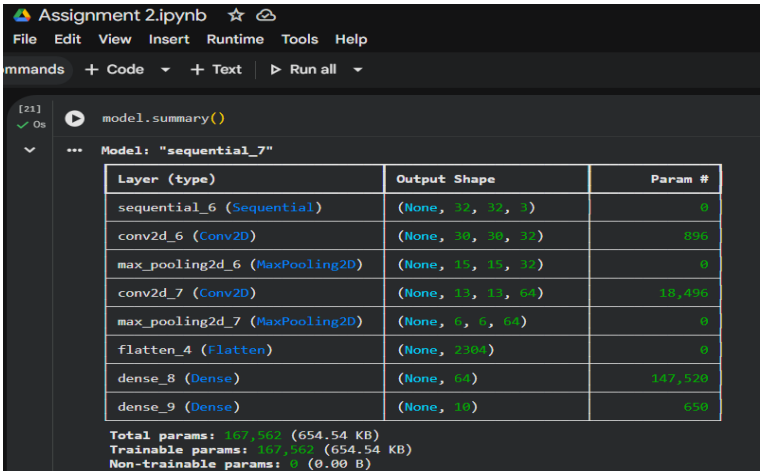
A custom CNN model was designed with the following layers:

- Conv2D (32 filters, ReLU activation)
- MaxPooling2D
- Conv2D (64 filters, ReLU activation)
- MaxPooling2D
- Flatten layer
- Dense layer (64 neurons, ReLU)
- Output layer (10 neurons, Softmax)

ReLU activation was used in hidden layers to introduce non-linearity, while Softmax activation was used in the output layer to generate probability distribution across 10 classes.

The model was compiled using:

- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Evaluation Metric: Accuracy



Layer (type)	Output Shape	Param #
sequential_6 (Sequential)	(None, 32, 32, 3)	0
conv2d_6 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_7 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_4 (Flatten)	(None, 2304)	0
dense_8 (Dense)	(None, 64)	147,520
dense_9 (Dense)	(None, 10)	650

Figure 2: Custom CNN Model Summary

Training of Custom CNN

The model was trained for 10 epochs with 20% validation split.

The gap between training and validation accuracy indicates slight overfitting but overall stable learning behavior.

Accuracy Curve

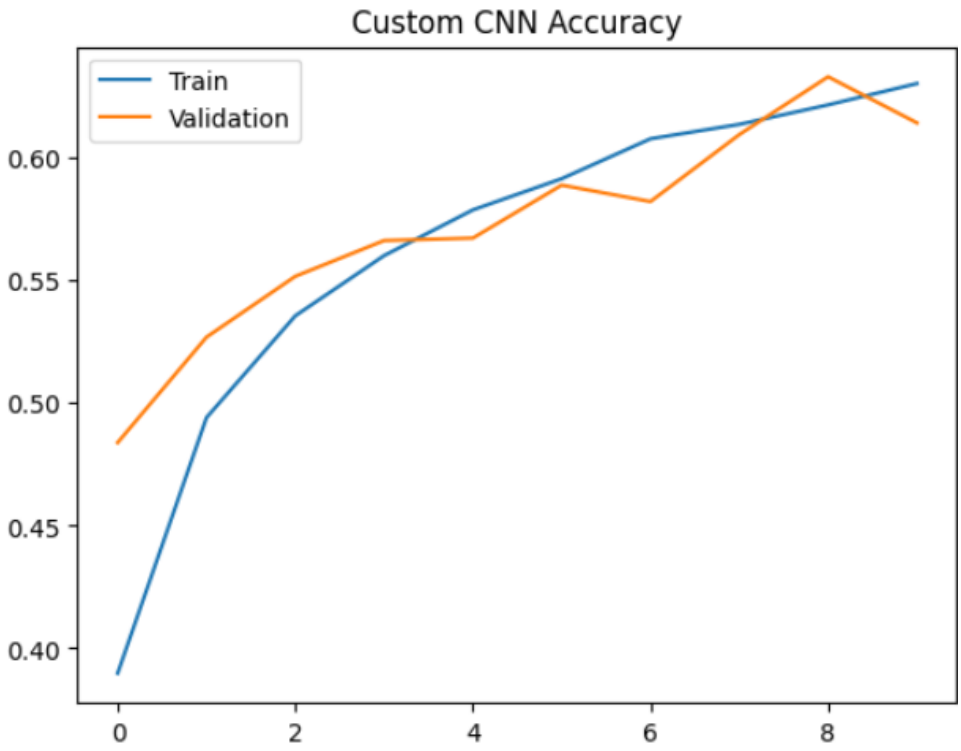


Figure 3: Custom CNN Accuracy Curve

Loss Curve

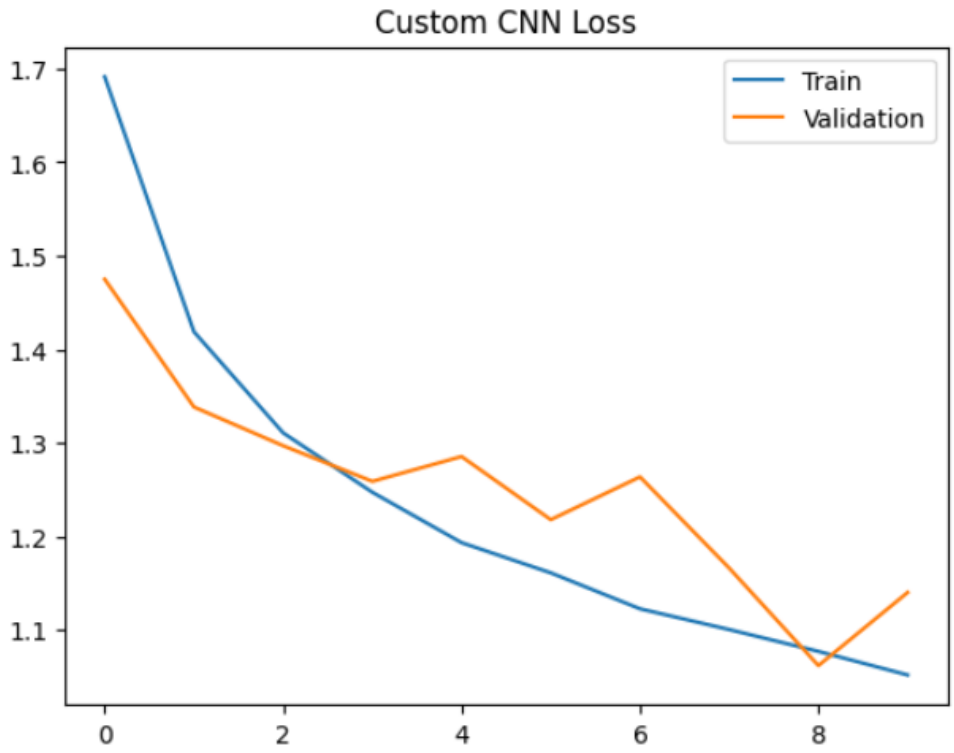


Figure 4: Custom CNN Loss Curve

The training accuracy improved steadily, while validation accuracy stabilized, indicating moderate generalization performance.

Evaluation Metrics (Custom CNN)

The model was evaluated on the test dataset.

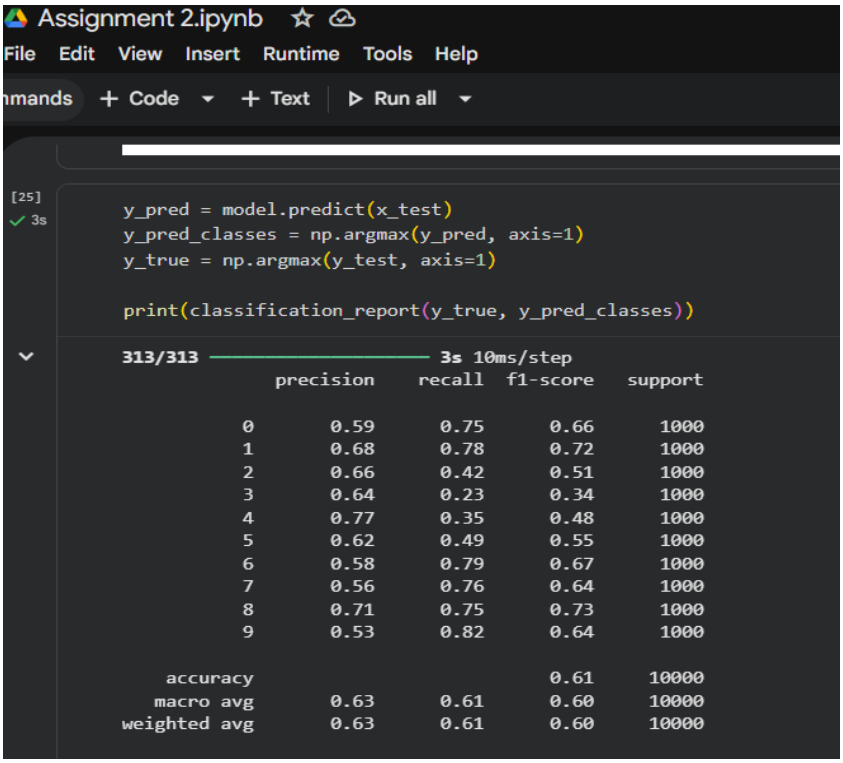


Figure 5: Classification Report of Custom CNN

The classification report shows balanced precision and recall across most classes, indicating good generalization capability of the model.

The overall test accuracy achieved was approximately 66%, demonstrating effective feature learning by the custom CNN.

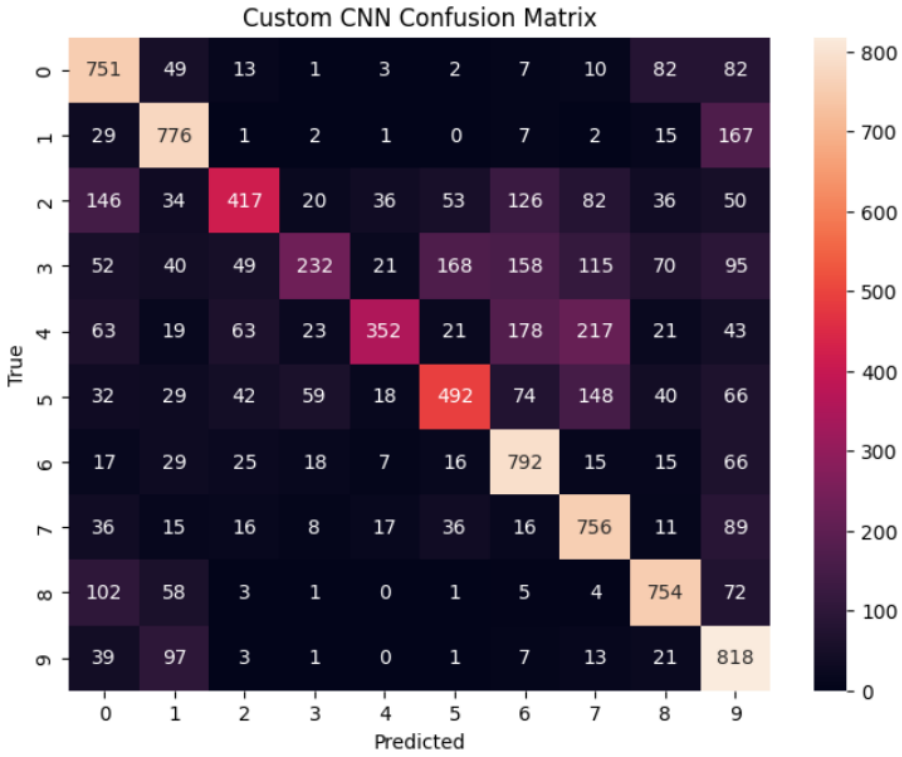


Figure 6: Confusion Matrix of Custom CNN

The confusion matrix shows correct predictions along the diagonal and misclassifications in off-diagonal entries.

Transfer Learning Using MobileNetV2

Transfer learning was implemented using a pretrained MobileNetV2 model trained on ImageNet.

The convolutional base was frozen, and custom dense layers were added on top for classification.

Figure 7: Transfer Learning Model Summary (MobileNetV2)

Transfer Learning Training Results

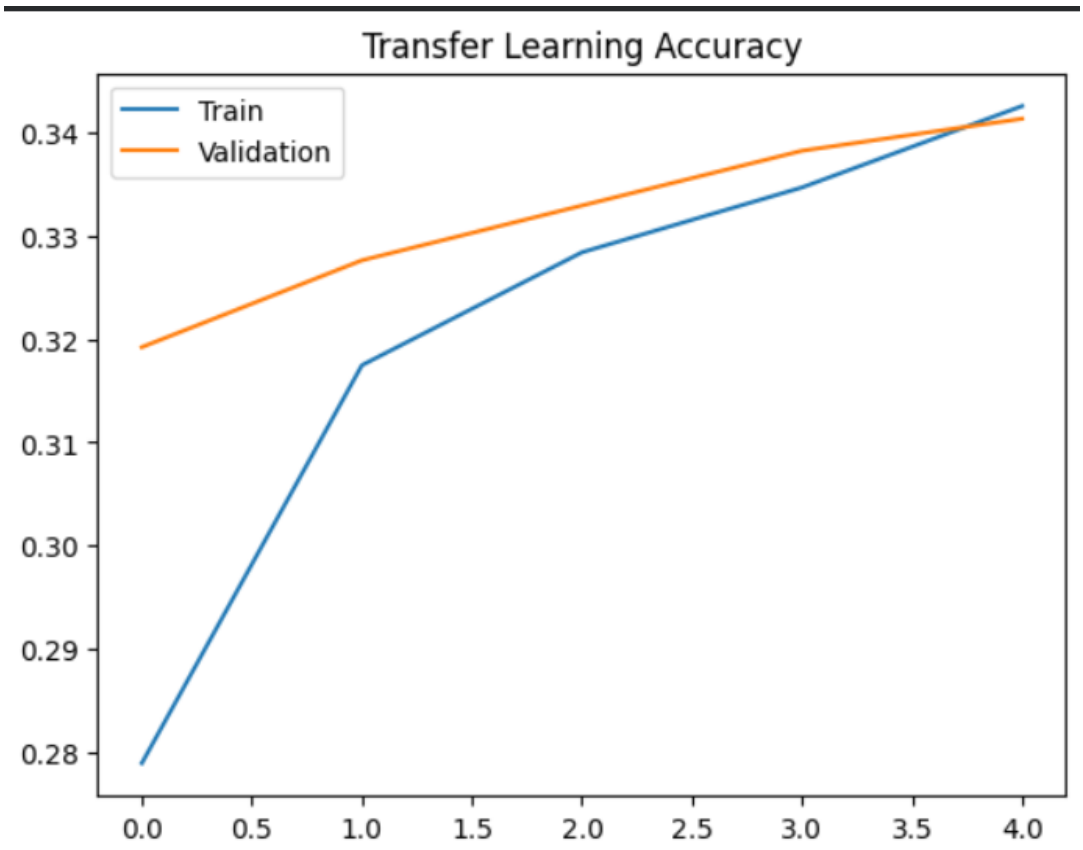


Figure 8: Transfer Learning Accuracy Curve

The pretrained model achieved lower accuracy compared to the custom CNN, possibly because MobileNetV2 is optimized for larger input images (224×224). Since CIFAR-10 images are 32×32, resizing and fine-tuning additional layers may improve performance.

Comparison of Models

Model	Test Accuracy
Custom CNN	66%
MobileNetV2	34%

The custom CNN performed better on CIFAR-10 dataset in this experiment.

Conclusion

In this experiment, both a custom CNN and a pretrained MobileNetV2 model were implemented for multiclass image classification. The custom CNN achieved higher accuracy on the CIFAR-10 dataset.

This experiment strengthened understanding of convolution, pooling, activation functions, transfer learning, and evaluation metrics. It demonstrated that model architecture and input size significantly influence performance.